

Murilo Tomaz Gonzaga
Prof. Douglas Rodrigues
Métodos Numéricos Computacionais
17 de maio de 2025

Relatório do Trabalho prático 3 - Sistemas Lineares

Implementação:

Foi utilizada a linguagem de programação Python para implementar a ideia do programa. Entre suas inúmeras bibliotecas, utilizei **matplotlib**, **gridspec**, **tabulate**, **numpy**, **math**, **time** e **os**, que deram suporte a toda a construção do projeto, desde a implementação da interface gráfica até a facilitação na implementação das funções.

A principal ideia do projeto era implementar métodos numéricos para a **resolução de sistemas lineares**, conforme aprendido em aula. Para isso, foram desenvolvidas as seguintes técnicas: **LU**, **Gauss-Compacto**, **Cholesky**, **Gauss-Jordan**, **Jacobi-Richardson** e **Gauss-Seidel**, permitindo obter as soluções de forma eficiente e estruturada.

Como solicitado na folha de requisição do projeto, todas essas implementações são gerenciadas por uma interface no CMD, com opções por meio de linhas de comando e escolha do usuário.

Imports e variáveis globais:

```
import os, time
import math
import matplotlib.pyplot as plt
import functions as functions
import numpy as np
import matplotlib.gridspec as gridspec
from tabulate import tabulate

A = np.array([[10, 2, -1], [-3, -6, 2], [1, 1, 5]], dtype=float)
B = np.array([27, -61.5, -21.5], dtype=float)
```

Como dito anteriormente, *import* de bibliotecas python que foram extremamente úteis para a resolução do trabalho, foram utilizados dois numpy arrays

Arquivo functions:

Utilitários:

```
## UTILS ##
def ver_pivos(A):
    n = len(A)
    for i in range(n):
        if A[i, i] == 0:
            return False
    return True

def ver_quadrada(A):
    lin, col = A.shape
    return lin == col

def ver_inversa(A):
    return np.linalg.det(A) != 0

def menores_principais(A, pos=False):
    n = len(A)
    for k in range(1, n + 1):
        sub = A[:k, :k]
        det = np.linalg.det(sub)
        if pos:
            if det <= 0:
                return False
            else:
                if det == 0:
                    return False
    return True
```

```
def ver_simetria(A):
    return np.array_equal(A, A.T)

def cri_linhas(A):
    n = len(A)
    X = np.zeros(n)
    for i in range(n):
        soma = 0
        for j in range(n):
            if i != j:
                soma += abs(A[i, j] / A[i, i])
        X[i] = soma
    return max(X) < 1

def cri_colunas(A):
    n = len(A)
    X = np.zeros(n)
    for j in range(n):
        soma = 0
        for i in range(n):
            if i != j:
                soma += abs(A[i, j] / A[j, j])
        X[j] = soma
    return max(X) < 1
| You, yesterday * aaaa ...

def estri_diagonal(A):
    n = len(A)
    for i in range(n):
        for j in range(n):
            if i != j:
                soma += abs(A[i, j])
            if abs(A[i, i]) <= soma:
                return False
    return True

def criterio_sassenfeld(A):
    n = len(A)
    beta = np.zeros(n, dtype=float)
    for i in range(n):
        soma1 = sum(abs(A[i, j]) * beta[j] for j in range(i))
        soma2 = sum(abs(A[i, j]) for j in range(i + 1, n))
        beta[i] = (soma1 + soma2) / abs(A[i, i])
    return np.all(beta < 1)
```

necessárias para a análise das matrizes envolvidas. Embora a biblioteca **numpy** ofereça suporte para diversas operações matriciais, alguns critérios de convergência dos métodos não estão diretamente implementados. Por isso, desenvolvi essas verificações manualmente, garantindo que os métodos numéricos possam ser aplicados corretamente.

Além disso, operações como cálculo de transposta, inversa e outras manipulações matriciais são facilitadas pela utilização do **numpy**, otimizando a execução do algoritmo.

Função solução:

```

## SOLUTION ##

def solution(L, U, B, tf, name):
    if not tf:
        return None

    n = len(B)

    if name in ["Decomposicao LU", "Gauss-Compacto", "Cholesky"]:
        Y = np.zeros(n)
        for i in range(n):
            Y[i] = B[i] - sum(L[i, j] * Y[j] for j in range(i))

        X = np.zeros(n)
        for i in range(n - 1, -1, -1):
            X[i] = (Y[i] - sum(U[i, j] * X[j] for j in range(i + 1, n))) / U[i, i]

    elif name in ["Jacobi-Richardson", "Gauss-Seidel"]:
        X = U
    else:
        return None

    return X

```

Em alguns métodos, a solução do sistema é obtida diretamente. No entanto, nos métodos de **Cholesky** e **LU**, o processo é realizado em duas etapas: a decomposição da matriz em duas componentes (L e U) e, em seguida, a resolução de dois sistemas lineares. Para padronizar o retorno em todos os métodos, criei uma função intermediária que realiza essa resolução, utilizando list comprehensions para otimizar o somatório em cada sistema, retornando diretamente o vetor solução.

Essa abordagem facilita a integração dos métodos, garantindo um retorno uniforme e permitindo a comparação direta dos resultados.

Métodos Numéricos:

Gauss-Jordan

```

## GAUSS-JORDAN ##
def gauss_jordan(A, B):
    if not ver_quadrada(A):
        return False, None
    elif not ver_pivos(A):
        return False, None
    elif not ver_inversa(A):
        return False, None

    n = len(B)
    B = B.reshape(-1, 1)
    M = np.hstack((A.astype(float), B.astype(float)))

    for i in range(n):
        M[i] = M[i] / M[i, i]
        for j in range(n):
            if i != j:
                M[j] = M[j] - M[j, i] * M[i]

    X = M[:, -1]

    return True, X

```

A implementação do método de **Gauss-Jordan** foi desenvolvida de maneira simples e direta, seguindo o pseudocódigo apresentado nos slides. Para isso, são utilizadas inicialmente funções utilitárias relacionadas aos critérios de convergência do método, garantindo que as condições necessárias sejam satisfeitas. Em seguida, aproveitam-se algumas funcionalidades oferecidas pela biblioteca **numpy**, como o método **len()** para obter a ordem da matriz e a função **np.hstack()** para criar a matriz aumentada. Essa abordagem facilita significativamente as operações necessárias para o método, visto que, sem o suporte da biblioteca, a montagem da matriz aumentada seria substancialmente mais complexa. Com a matriz devidamente organizada, o processo de eliminação é implementado conforme descrito no pseudocódigo, resultando em uma solução clara e aderente aos conceitos teóricos

Decomposição LU

```
## DECOMPOSIÇÃO LU ##
def decomposicao_lu(A):
    # convergencia dos amigos
    if not ver_quadrada(A):
        return False, None, None

    if not menores_principais(A, False):
        return False, None, None

    n = len(A)
    L = np.identity(n, dtype=float)
    U = np.zeros((n, n), dtype=float)

    for k in range(n):
        for j in range(k, n):
            soma = 0
            for s in range(k):
                soma += L[k, s] * U[s, j]

            U[k, j] = A[k, j] - soma

        for i in range(k + 1, n):
            soma = 0
            for s in range(k):
                soma += L[i, s] * U[s, k]

            L[i, k] = (A[i, k] - soma) / U[k, k]

    return True, L, U
```

A implementação do método de **Decomposição LU** foi desenvolvida de maneira clara e estruturada, seguindo os critérios de convergência necessários para garantir sua aplicabilidade. Inicialmente, é verificado se a matriz é quadrada e se os menores principais são diferentes de zero, condições essenciais para que o processo de decomposição seja válido. Em seguida, são criadas duas matrizes: LLL, que é

inicializada como a matriz identidade, e U, que é inicialmente preenchida com zeros. A etapa de decomposição é realizada através de dois loops aninhados. No primeiro loop, são calculados os elementos da matriz U a partir da soma dos produtos entre os elementos de L e U, seguindo a expressão do método. No segundo loop, os elementos de L são determinados com base nos valores de A, levando em conta as contribuições acumuladas das iterações anteriores. Ao final do processo, as matrizes L e U representam, respectivamente, a matriz triangular inferior e a matriz triangular superior que, multiplicadas, reconstituem a matriz original. Essa abordagem permite uma solução eficiente de sistemas lineares, simplificando o processo de substituição progressiva e retroativa.

Cholesky

```
def cholesky(A):
    # convergencia dos amigos
    if not ver_simetria(A):
        return False, None

    elif not menores_principais(A, True):
        return False, None

    n = len(A)
    L = np.zeros((n, n), dtype=float)
    for i in range(n):
        for j in range(i):
            soma = 0.0
            for k in range(j):
                soma += L[i, k] * L[j, k]
            if i == j:
                L[i, j] = math.sqrt(A[i, i] - soma)
            else:
                L[i, j] = (A[i, j] - soma) / L[j, j]
    return True, L
```

A implementação do método de **Decomposição de Cholesky** foi desenvolvida de maneira estruturada, garantindo a verificação dos critérios necessários para sua aplicação. Inicialmente, é checada a simetria da matriz e a positividade dos menores principais, condições indispensáveis para que a decomposição seja possível. Caso os critérios sejam atendidos, a matriz L é criada e inicializada com zeros, representando a matriz triangular inferior resultante do processo. Ao final da execução, a decomposição

retorna a matriz L, de forma que o produto $L * L^T$ reconstitui a matriz original. Esse método se destaca por ser mais eficiente em termos computacionais do que a decomposição LU para matrizes simétricas e definidas positivas.

Gauss-Compacto

```
## GAUSS-COMPACTO ##
def gauss_compacto(A):
    if not ver_quadrada(A):
        return False, None
    elif not ver_pivos(A):
        return False, None
    elif not ver_inversa(A):
        return False, None

    n = len(A)
    L = np.eye(n)
    U = A.copy()
    for i in range(n):
        for j in range(i + 1, n):
            if U[i, i] == 0:
                return False, None
            else:
                L[j, i] = U[j, i] / U[i, i]
                U[j, i:] = U[j, i:] - L[j, i] * U[i, i:]
    return True, L, U
```

A implementação do método de **Gauss-Compacto** foi estruturada para realizar a decomposição de uma matriz quadrada em duas componentes: a matriz triangular inferior L e a matriz triangular superior U. Primeiramente, são verificadas as condições de aplicabilidade do método, garantindo que a matriz seja quadrada, possua pivôs não nulos e seja invertível. Uma vez satisfeitos esses critérios, a matriz L é inicializada como a matriz identidade e a matriz U é uma cópia da matriz original. Durante o processo de eliminação, os elementos abaixo da diagonal principal são zerados por meio de operações lineares, sendo seus valores armazenados em L. Ao final, as matrizes L e U representam a decomposição desejada, permitindo a resolução do sistema linear de forma mais eficiente.

Jacobi-Richardson

```
def jacobi_richardson(A, B):
    if not cri_linhas(A) or not cri_colunas(A):
        return False, None, None, None

    n = len(A)
    k_max = 50
    err = 1e-5
    X = np.zeros(n, dtype=float)
    h = []

    def calcula_linha(A, B, X_, i, results):
        soma = 0.0
        for j in range(n):
            if j != i:
                soma += A[i, j] * X_[j]
        results[i] = (B[i] - soma) / A[i, i]

    for k in range(k_max):
        X_ = X.copy()

        threads = []
        results = np.zeros(n, dtype=float)

        for i in range(n):
            thread = threading.Thread(target=calcula_linha, args=(A, B, X_, i, results))
            threads.append(thread)
            thread.start()

        for thread in threads:
            thread.join()

        X = results
        h.append(X.copy())

        erro_relativo = np.linalg.norm(X - X_, ord=np.inf) / np.linalg.norm(X, ord=np.inf)

        if erro_relativo < err:
            return True, X, k + 1, np.array(h), erro_relativo

    erro_relativo = np.linalg.norm(X - X_, ord=np.inf) / np.linalg.norm(X, ord=np.inf)
    return False, X, k_max, np.array(h), erro_relativo
```

A implementação do método de **Jacobi-Richardson** foi organizada para resolver sistemas lineares de forma iterativa. Inicialmente, verificam-se os critérios de convergência, garantindo que a matriz seja adequada para o método. Definimos um número máximo de iterações (`k_max = 50`) e um erro máximo aceitável (`err = 1e-5`). Em seguida, inicializamos o vetor solução `X` com zeros e criamos uma lista `h` para armazenar o histórico das aproximações em cada iteração.

Para facilitar o cálculo das linhas de forma paralela, é definida uma função interna chamada `calcula_linha`, que realiza a operação de somatório para cada linha da matriz. Essa função é chamada em threads separadas para cada linha, permitindo que as operações sejam realizadas em paralelo e aceleram o processo iterativo.

Em cada iteração, o vetor solução `X` é atualizado com os valores calculados, e o erro relativo é avaliado. Caso o erro seja menor que o limite estabelecido, o método é

considerado convergente, e o processo é encerrado, retornando o vetor solução, o número de iterações realizadas, o histórico de aproximações e o erro final. Caso o número máximo de iterações seja alcançado sem convergência, o método retorna o estado atual da solução, indicando que não houve convergência.

Essa abordagem paralelizada permite que o método de Jacobi-Richardson seja mais eficiente em relação a uma implementação sequencial, aproveitando melhor o processamento da máquina.

Gauss-Seidel

```
## Gauss-Seidel ##
def gauss_seidel(A, B):
    if not cri_linhas(A) or not criterio_sassenfeld(A):
        return False, None, None, None

    n = len(A)
    k_max = 50
    err = 1e-5

    X = np.zeros(n, dtype=float)
    h = []

    for k in range(k_max):
        X_ = X.copy()

        for i in range(n):
            soma1 = 0.0
            soma2 = 0.0

            for j in range(i):
                soma1 += A[i, j] * X[j]

            for j in range(i + 1, n):
                soma2 += A[i, j] * X_[j]

            X[i] = (B[i] - soma1 - soma2) / A[i, i]

        h.append(X.copy())

        erro_relativo = np.linalg.norm(X - X_, ord=np.inf) / np.linalg.norm(X, ord=np.inf)

        if erro_relativo < err:
            return True, X, k + 1, np.array(h), erro_relativo

    erro_relativo = np.linalg.norm(X - X_, ord=np.inf) / np.linalg.norm(X, ord=np.inf)
    return False, X, k_max, np.array(h), erro_relativo
```

A implementação do método de **Gauss-Seidel** verifica inicialmente os critérios de convergência, garantindo que a matriz atenda ao critério das linhas e ao critério de Sassenfeld. Em seguida, define-se o número máximo de iterações ($k_{\text{max}} = 50$), o erro tolerável ($\text{err} = 1e-5$) e inicia-se o vetor solução X com zeros. Durante as iterações, cada componente de X é atualizado imediatamente após seu cálculo, aproveitando os valores mais recentes, o que acelera a convergência em comparação ao método de Jacobi. Ao final de cada iteração, o erro relativo é calculado e, caso seja menor que o limite definido, o processo é encerrado com sucesso. Caso contrário, ele retorna ao estado atual após o número máximo de iterações.

Retorno:

Nos métodos iterativos fazemos um vetor com os valores de x para cada iteração para que possamos utilizar nos gráficos e tabelas.

Funções Gráficas:

```
def plot_grafico1(hist_jacobi, it_jacobi, hist_seidel, it_seidel):
    n_vars_jacobi = hist_jacobi.shape[1]
    n_vars_seidel = hist_seidel.shape[1]

    mpt.figure(figsize=(10, 6))

    marcadores = ["o", "s", "^"]
    cores_jacobi = ["#1f77b4", "#ff7f0e", "#2ca02c"]
    cores_seidel = ["#d62728", "#9467bd", "#8c564b"]

    for i in range(n_vars_jacobi):
        mpt.plot(
            range(it_jacobi),
            hist_jacobi[:, it_jacobi, i],
            marker=marcadores[i],
            linestyle="--",
            color=cores_jacobi[i],
            label=f"Jacobi x{i+1}",
        )

    for i in range(n_vars_seidel):
        mpt.plot(
            range(it_seidel),
            hist_seidel[:, it_seidel, i],
            marker=marcadores[i],
            linestyle="--",
            color=cores_seidel[i],
            label=f"Gauss-Seidel x{i+1}",
        )

    mpt.xlabel("Iterações")
    mpt.ylabel("Valores de X")
    mpt.title("Convergência dos Métodos Iterativos")
    mpt.legend()
    mpt.grid(True)
    min_y = int(min(hist_jacobi.min(), hist_seidel.min())) - 1
    max_y = int(max(hist_jacobi.max(), hist_seidel.max())) + 1
    mpt.yticks(range(min_y, max_y + 1, 1))
    mpt.tight_layout()
    mpt.savefig("grafico_convergencia.png")
```

```
def plot_grafico2(it_jacobi, it_seidel):
    mpt.figure(figsize=(6, 6))
    metodos = ["Jacobi-Richardson", "Gauss-Seidel"]
    iteracoes = [it_jacobi, it_seidel]

    mpt.bar(metodos, iteracoes, color="#5B9BD5")
    mpt.ylabel("Número de Iterações")
    mpt.title("Número de Iterações por Método")
    mpt.tight_layout()
    mpt.savefig("grafico_iteracoes.png")
```

```
def plot_tabela(X_lu, X_gc, X_c, X_gj, X_jacobi, X_seidel, tempos, erros):
    metodos = [
        "LU",
        "Gauss-Compacto",
        "Cholesky",
        "Gauss-Jordan",
        "Jacobi-Richardson",
        "Gauss-Seidel",
    ]
    solucoes = [X_lu, X_gc, X_c, X_gj, X_jacobi, X_seidel]

    def formatar_solucao(solucacao):
        if solucacao is None or solucacao is False:
            return ["Não Aplicável"] * 3
        try:
            return [f"x{i+1} = {v:.6g}" for i, v in enumerate(solucacao)]
        except Exception:
            return [str(solucacao)] * 3

    table_data = []
    for i, (metodo, solucacao, tempo) in enumerate(zip(metodos, solucoes, tempos)):
        formatted_sol = formatar_solucao(solucacao)

        if metodo in ["Jacobi-Richardson", "Gauss-Seidel"]:
            erro_str = f"{erros[i]:.2e}" if erros[i] is not None else "N/A"
        else:
            erro_str = "N/A"

        row = [metodo] + formatted_sol + [f"{tempo:.2e}s"] + [erro_str]
        table_data.append(row)

    fig = mpt.figure(figsize=(12, 4))
    spec = gridspec.GridSpec(nrows=1, ncols=1)
    ax = fig.add_subplot(spec[0])
    ax.axis("off")
    ax.set_title("Soluções Obtidas", fontsize=14, pad=10)

    col_labels = ["Método", "x1", "x2", "x3", "Tempo de exec.", "Erro Relativo"]
    tabela = ax.table(
        cellText=table_data, colLabels=col_labels, loc="center", cellLoc="left"
    )

    tabela.auto_set_font_size(False)
    tabela.set_fontsize(10)
    tabela.auto_set_column_width(col=list(range(len(col_labels))))
    tabela.scale(1.5, 2)

    mpt.tight_layout()
    mpt.savefig("tabela_solucoes.png")
```

Na primeira função realizamos um gráfico contínuo com os valores de x_1 , x_2 , x_3 por iteração, para compararmos os resultados, dos métodos iterativos,

Na segunda função é basicamente um gráfico de barras, do número de iterações por método iterativo para verificarmos a discrepância entre o número de iterações.

Na terceira função, fiz uma tabela completa para realizarmos a análise de todos os métodos, com número de iterações, tempo de execução, valor do vetor resolução e erro relativo.

Função Main:

```
def main():
    while True:
        e = select()
        if e == 0:
            os.system('cls' if os.name == 'nt' else 'clear')
            break
        else:
            exec(e)
```

Na função `main`, é realizado um reprocessamento simples para que o usuário possa utilizar o programa várias vezes, sendo interrompido apenas quando a opção de "Sair" for selecionada. É importante mencionar que, ao escolher a opção 7, a função `exec(0)` também finaliza o reprocessamento, encerrando a execução do programa.

Função select:

```
def select():
    print("Deseja resolver o problema com: ")
    print("[1] - Decomposicao LU")
    print("[2] - Gauss-Compacto")
    print("[3] - Cholesky")
    print("[4] - Gauss-Jordan")
    print("[5] - Jacobi-Richardson")
    print("[6] - Gauss-Seidel")
    print("[7] - Comparar métodos")
    print("[0] - Sair")

    while True:
        try:
            e = int(input("Digite o index desejado: "))
            if 0 ≤ e ≤ 7:
                return e
            else:
                print("Digite um numero entre 0 e 7")
        except ValueError:
            print("Digite um numero válido")
```

A função responsável pela interface de linha de comando foi implementada conforme solicitado no PDF do trabalho. Além disso, foi incluída uma verificação utilizando um bloco `try-except` para evitar entradas inválidas, como números fora do intervalo esperado ou letras, garantindo que o usuário insira apenas valores válidos. Dessa forma, o programa se torna mais robusto e amigável, evitando erros inesperados durante a execução.

Função exec:

```
def exec(metodo=None):
    tempos = [0, 0, 0, 0, 0, 0]
    erros = [None, None, None, None, None, None]

    X_lu = X_gc = X_c = X_gj = X_jacobi = X_seidel = None
    hist_jacobi = None
    hist_seidel = None
    it_jacobi = it_seidel = 0

    print()

    if metodo is None or metodo == 1 or metodo == 7:
        start = time.time()
        tf, L, U = functions.decomposicao_lu(A)
        X_lu = functions.solution(L, U, B, tf, "Decomposicao LU")
        tempos[0] = time.time() - start

    if metodo == 2 or metodo == 7:
        start = time.time()
        tf, L, U = functions.gauss_compacto(A)
        X_gc = functions.solution(L, U, B, tf, "Gauss-Compacto")
        tempos[1] = time.time() - start

    if metodo == 3 or metodo == 7:
        start = time.time()
        tf, L = functions.cholesky(A)
        if tf:
            X_c = functions.solution(L, L.T, B, tf, "Cholesky")
        else:
            if metodo == 3:
                print("Cholesky não pôde ser aplicado por não atender aos critérios.")
            X_c = None
        tempos[2] = time.time() - start

    if metodo == 4 or metodo == 7:
        tf, X_gj = functions.gauss_jordan(A, B)

    if metodo == 5 or metodo == 7:
        start = time.time()
        tf, X, it_jacobi, hist_jacobi, erros[4] = functions.jacobi_richardson(A, B)
        X_jacobi = functions.solution(None, X, B, tf, "Jacobi-Richardson")
        tempos[4] = time.time() - start

    if metodo == 6 or metodo == 7:
        start = time.time()
        tf, X, it_seidel, hist_seidel, erros[5] = functions.gauss_seidel(A, B)
        X_seidel = functions.solution(None, X, B, tf, "Gauss-Seidel")
        tempos[5] = time.time() - start

    if metodo == 7:
        plot_tabela(X_lu, X_gc, X_c, X_gj, X_jacobi, X_seidel, tempos, erros)
        plot_grafico1(hist_jacobi, it_jacobi, hist_seidel, it_seidel)
        plot_grafico2(it_jacobi, it_seidel)
```

```

os.system('cls' if os.name == 'nt' else 'clear')
print("Obrigado por utilizar")
exit(1)
else:
    metodos = [
        "Decomposicao LU",
        "Gauss-Compacto",
        "Cholesky",
        "Gauss-Jordan",
        "Jacobi-Richardson",
        "Gauss-Seidel",
    ]
    solucoes = [X_lu, X_gc, X_c, X_gj, X_jacobi, X_seidel]
    i = metodo - 1
    solucao = solucoes[i]
    tempo = tempos[i]
    erro = erros[i] if i ≥ 4 else None
    print(f"Resultado do metodo {metodos[i]}:")
    if solucao is None:
        print("Solução: N/A")
    else:
        table_data = []
        for idx, val in enumerate(solucao):
            table_data.append([f"x{idx+1}", f"{val:.6g}"])

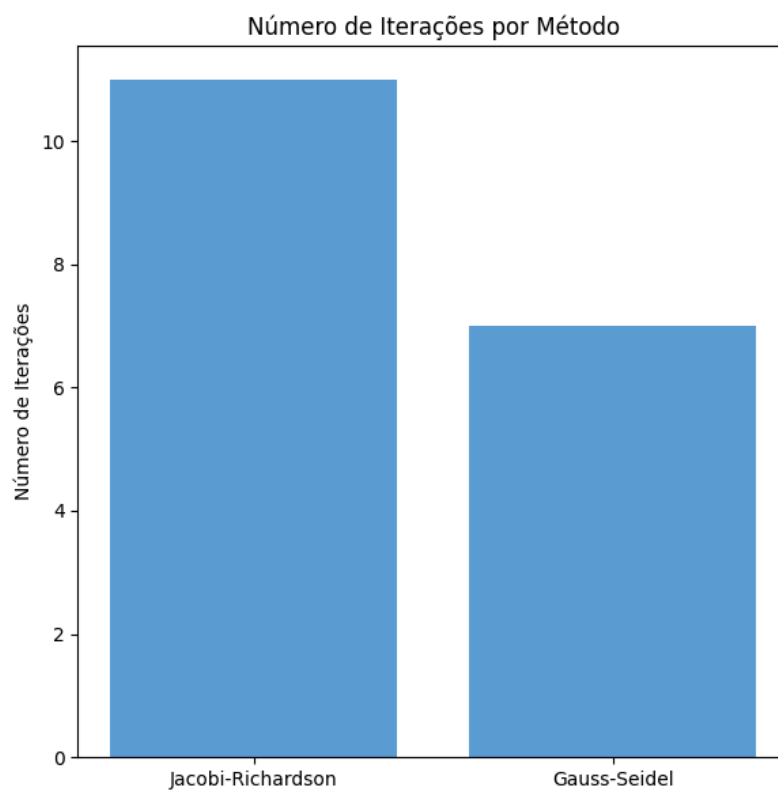
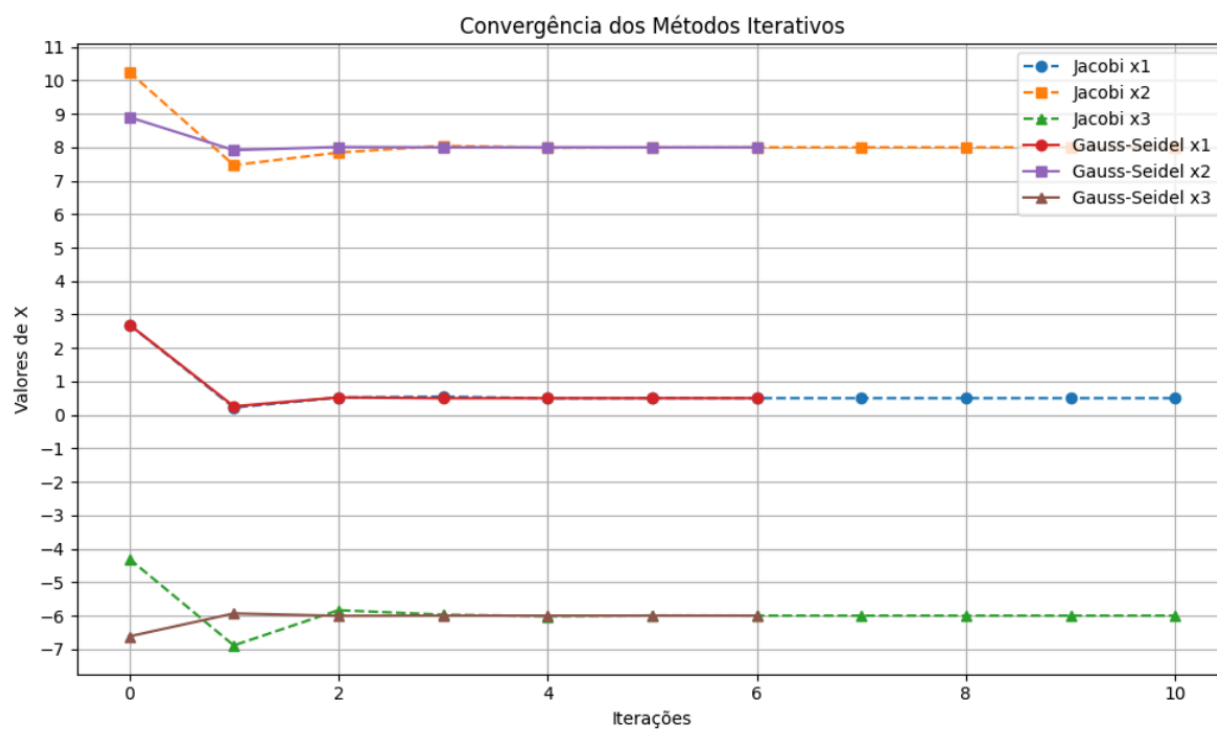
        table_data.append(["Tempo de execucao", f"{tempo:.2e}s"])
        if erro is not None:
            table_data.append(["Erro relativo", f"{erro:.2e}"])
        else:
            table_data.append(["Erro relativo", "N/A"])
        print(tabulate(table_data, headers=["Variavel", "Valor"], tablefmt="simple"))
    time.sleep(4)
    os.system('cls' if os.name == 'nt' else 'clear')

```

De acordo com o número selecionado pelo usuário, a execução é direcionada para a função `exec`, que contém a lógica principal do programa. Nessa função, são utilizados os métodos definidos no arquivo `functions`. Recebemos as matrizes, a solução, o número de iterações e o erro. Durante a execução de cada método, calculamos o tempo de processamento utilizando a biblioteca `time`. Para organizar o fluxo, a lógica é segmentada em blocos `if`, correspondentes a cada valor de `e` selecionado. Ao final da execução, utilizamos a biblioteca `tabulate` para exibir os resultados formatados, facilitando a visualização do método aplicado ao problema. No entanto, se o usuário escolher `e == 7`, que corresponde à comparação de resultados, todos os métodos são executados sequencialmente, e, ao término, utilizamos funções gráficas para gerar tabelas e gráficos representativos dos resultados obtidos.

Análises Gráficas:

Métodos iterativos:



Todos os Métodos:

Soluções Obtidas

Método	x ₁	x ₂	x ₃	Tempo de exec.	Erro Relativo
LU	x ₁ = 0.5	x ₂ = 8	x ₃ = -6	2.14e-04s	N/A
Gauss-Compacto	x ₁ = 0.5	x ₂ = 8	x ₃ = -6	5.39e-05s	N/A
Cholesky	Não Aplicável	Não Aplicável	Não Aplicável	4.17e-05s	N/A
Gauss-Jordan	x ₁ = 0.5	x ₂ = 8	x ₃ = -6	0.00e+00s	N/A
Jacobi-Richardson	x ₁ = 0.499997	x ₂ = 7.99999	x ₃ = -6.00001	5.49e-03s	3.10e-06
Gauss-Seidel	x ₁ = 0.500003	x ₂ = 8	x ₃ = -6	1.70e-04s	4.05e-06

No gráfico de convergência dos métodos iterativos, verificamos que os métodos foram bem parecidos em termos de aproximação dos valores de xxx a cada iteração. Nesse caso, é nítido que o método de Jacobi se mostrou um pouco mais constante em x₂, enquanto o método de Gauss-Seidel apresentou um desempenho mais estável em x₃. Em relação a x₁, ambos os métodos seguiram trajetórias praticamente idênticas, de acordo com a escala apresentada. No entanto, ao analisarmos em conjunto com o gráfico do número de iterações por método, observamos que o método de Jacobi demandou mais iterações para ajustar os valores finais, totalizando 12 iterações, enquanto o método de Gauss-Seidel necessitou de apenas 9 iterações para alcançar a solução desejada.

Ao analisarmos a tabela de soluções obtidas, podemos identificar que os métodos diretos (LU, Gauss-Compacto e Gauss-Jordan) convergiram para a solução exata do sistema de forma precisa, sem apresentar erro relativo, uma vez que o método é determinístico. No entanto, o método de Cholesky não foi aplicável, possivelmente devido à falta de simetria positiva definida na matriz. Em contrapartida, os métodos iterativos (Jacobi-Richardson e Gauss-Seidel) também conseguiram aproximar bem os valores, mas com pequenas diferenças em relação aos métodos diretos, refletidas nos erros relativos. Observa-se ainda que o método de Jacobi-Richardson necessitou de um tempo maior para a execução, possivelmente devido ao número de iterações necessárias para estabilização dos valores, enquanto o método de Gauss-Seidel foi mais rápido e demandou menos iterações. Essas

observações indicam que, para este problema, Gauss-Seidel se mostrou mais eficiente entre os métodos iterativos, tanto em termos de iterações quanto de tempo de execução.

CONCLUSÃO

Com a realização deste trabalho, ficou nítida a importância dos métodos computacionais para a resolução de sistemas lineares. A implementação desses métodos representa um desafio inicial, mas, com um algoritmo bem estruturado e uma compreensão clara dos conceitos, o processo se torna mais acessível e eficaz. Além disso, a proposta de aplicar diferentes algoritmos para resolver sistemas lineares variados enriquece a prática e contribui significativamente para o aprendizado, permitindo uma imersão mais profunda nos métodos numéricos e na linguagem Python. Essa experiência reforça não apenas o entendimento teórico, mas também a capacidade de análise crítica sobre a eficiência e aplicabilidade de cada método em diferentes cenários.